

Experiment 3: Loan Sanction Amount Prediction using Linear Regression Models

1 Aim and Objective

The aim of this experiment is to build and compare regression models that predict the **Loan Sanction Amount (USD)** for applicants, given demographic, financial, and property-related attributes. The objectives are to:

- Perform exploratory data analysis (EDA) on the loan dataset.
- Preprocess the data (handle missing values, encode categorical variables, scale features).
- Train a baseline Linear Regression model.
- Train regularized variants – Ridge, Lasso, and Elastic Net regression.
- Tune hyperparameters using 5-fold cross-validation.
- Evaluate and compare all models using standard regression metrics.
- Analyze overfitting/underfitting and the bias–variance trade-off.

2 Dataset Description

The dataset used is a loan prediction dataset split into `train.csv` and `test.csv`. The training set contains 30,000 records and 24 columns, described below.

- **Identifiers:** Customer ID, Name, Property ID
- **Demographics:** Gender, Age, Dependents
- **Financial attributes:** Income (USD), Income Stability, Profession, Type of Employment, Current Loan Expenses (USD), Credit Score, No. of Defaults, Has Active Credit Card
- **Loan attributes:** Loan Amount Request (USD), Expense Type 1, Expense Type 2
- **Property attributes:** Property Age, Property Type, Property Location, Property Price, Co-Applicant, Location
- **Target variable:** Loan Sanction Amount (USD) – present only in `train.csv`; this is the value predicted for `test.csv`.

Several columns contain missing values, most notably Income (USD) (4,576 missing), Type of Employment (7,270 missing), Dependents (2,493 missing), Property Age (4,850 missing), Credit Score (1,703 missing), and the target column itself (340 missing). The target variable is right-skewed (skewness ≈ 1.24), with a mean of \$47,508 and a median of \$35,209.

3 Preprocessing Steps

The following preprocessing pipeline was applied before model training:

1. **Column removal:** `Customer ID` and `Name` were dropped, as they are unique identifiers with no predictive value.
2. **Target row filtering:** Rows with a missing target value (`Loan Sanction Amount (USD)`) were removed, since they cannot be used for supervised training.
3. **Categorical encoding:** All categorical columns (`Gender`, `Income Stability`, `Profession`, `Type of Employment`, `Location`, `Expense Type 1/2`, `Has Active Credit Card`, `Property Location`) were one-hot encoded using `pandas.get_dummies`, with the first category dropped to avoid multicollinearity.
4. **Missing value imputation:** Remaining missing numeric values were imputed using the median of each column.
5. **Train/validation split:** The training data was split 80/20 into training and validation sets (`random_state=42`), since `test.csv` has no labels and cannot be used for local evaluation.
6. **Feature scaling:** All features were standardized (zero mean, unit variance) using `StandardScaler`, fit on the training set and applied to both training and validation sets. This step was necessary because the raw features span very different scales (e.g. `Loan Amount Request` in the tens of thousands vs. binary dummy columns), which previously caused an ill-conditioned matrix warning in Ridge regression and convergence warnings in Lasso/Elastic Net.

4 Implementation Details

All models were implemented using `scikit-learn`. The workflow consisted of:

- **Baseline model:** `LinearRegression` fit on the scaled training data.
- **Regularized models:** `Ridge`, `Lasso`, and `ElasticNet`, each fit with default parameters first, then re-tuned.
- **Hyperparameter tuning:** `GridSearchCV` with 5-fold cross-validation and `neg_root_mean_squared_error` scoring was used to search:
 - Ridge: $\alpha \in \{0.01, 0.1, 1, 10, 100\}$
 - Lasso: $\alpha \in \{0.01, 0.1, 1, 10, 100\}$
 - Elastic Net: $\alpha \in \{0.01, 0.1, 1, 10\}$, `l1_ratio` $\in \{0.2, 0.5, 0.8\}$
- **Evaluation:** Each model's RMSE and R^2 were computed on the held-out validation set, and training RMSE was also recorded for overfitting analysis.

A representative code snippet for the hyperparameter tuning step is shown below:

```
ridge_grid = GridSearchCV(Ridge(), {'alpha': [0.01, 0.1, 1, 10, 100]},
                           cv=5, scoring='neg_root_mean_squared_error')
ridge_grid.fit(X_train, y_train)
ridge_best = ridge_grid.best_estimator_
```

The best hyperparameters found were:

- Ridge: $\alpha = 100$
- Lasso: $\alpha = 100$
- Elastic Net: $\alpha = 0.1$, $l1_ratio = 0.5$

5 Visualizations

Figure 1 presents all six required visualizations in a single combined figure: the target variable distribution, a feature-vs-target scatter plot (Loan Amount Request vs. Loan Sanction Amount), predicted-vs-actual values, the residual plot, training-vs-validation error across models, and a coefficient comparison bar plot across all four models.

6 Performance Tables

Table 1: Validation performance of all trained models

Model	RMSE	R^2
Linear Regression	31,834.58	0.5535
Ridge	31,837.77	0.5534
Lasso	31,834.05	0.5535
Elastic Net	31,889.06	0.5520

All four models perform almost identically, indicating that regularization has little effect on this dataset once features are properly scaled – the relationship between features and target is close to linear, and multicollinearity/overfitting are not severe issues here.

7 Overfitting and Underfitting Analysis

Training and validation RMSE were compared for each model (see the bottom-left panel of Figure 1). Across all four models, training and validation error are very close to one another, with no large gap between the two. This indicates that:

- The models are **not overfitting** – if they were, training error would be substantially lower than validation error.
- The models show mild **underfitting**, since the overall R^2 (≈ 0.55) leaves a large portion of variance in the target unexplained. A plain linear model may be too simple to capture the full relationship between the features and the loan sanction amount.

8 Bias–Variance Analysis

The near-identical performance of the unregularized Linear Regression model and the regularized Ridge/Lasso/Elastic Net models (even at fairly strong regularization strengths, e.g. Ridge/Lasso $\alpha = 100$) suggests:

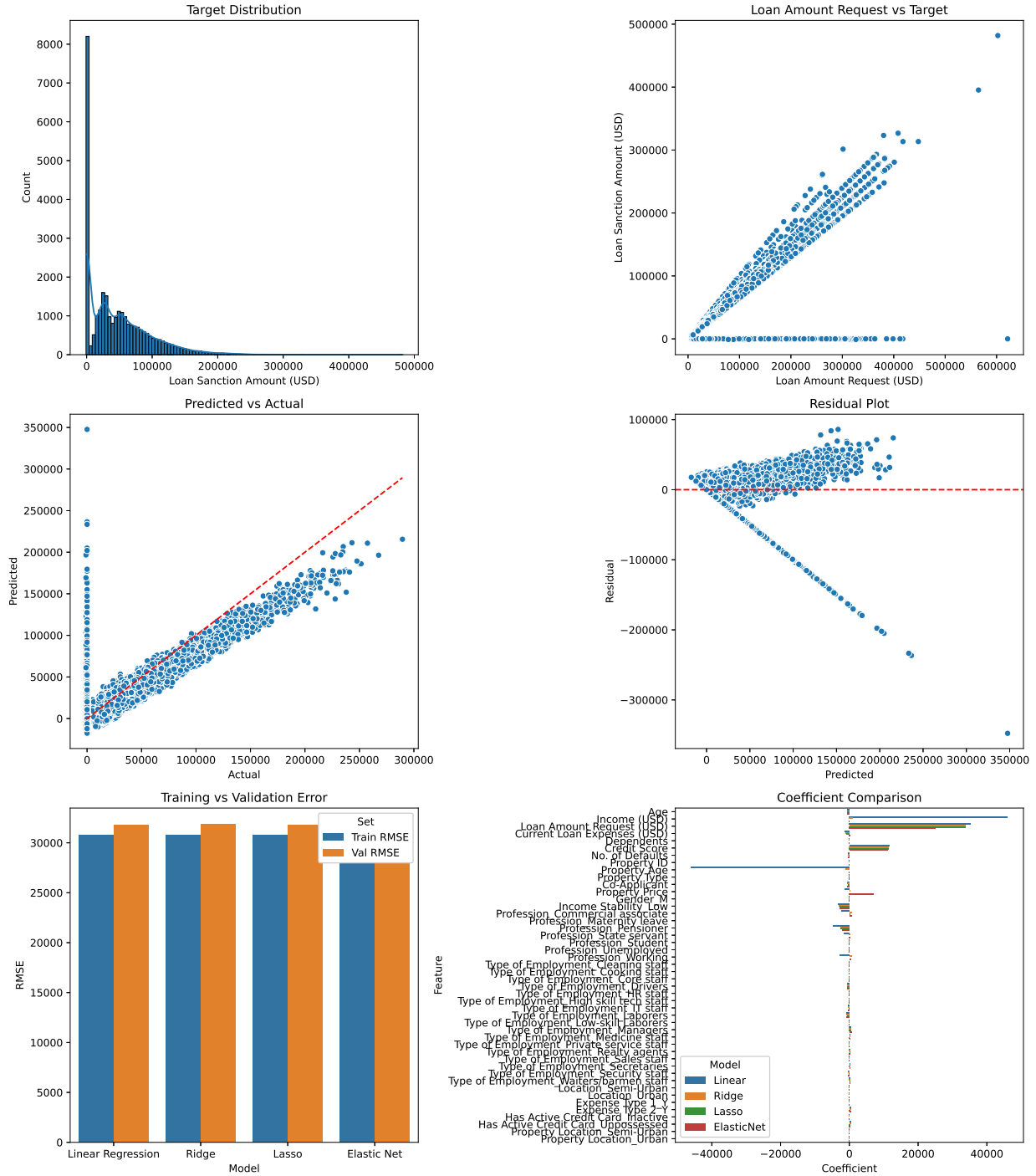


Figure 1: Combined visualization panel: (top-left) target distribution, (top-right) feature vs. target scatter, (middle-left) predicted vs. actual, (middle-right) residuals, (bottom-left) training vs. validation error, (bottom-right) coefficient comparison across models.

- **High bias, low variance:** All models converge to similar, moderate error, meaning the models are stable across regularization strengths (low variance) but are limited by the linear model's inability to fully capture the underlying relationship (higher bias).

- Regularization did not meaningfully reduce validation error compared to the baseline, implying that variance was not the dominant source of error to begin with – the primary limitation is model bias/capacity rather than overfitting to noise.

9 Observations and Conclusion

- Feature scaling was essential: without it, Ridge produced an ill-conditioned matrix warning and Lasso/Elastic Net failed to converge, due to the large differences in feature magnitudes.
- Loan Amount Request (USD) appears to be one of the strongest predictors of the target, consistent with domain intuition (higher requested amounts tend to correlate with higher sanctioned amounts).
- All four models achieved comparable RMSE ($\approx \$31,800$ – $\$31,900$) and R^2 (≈ 0.55), suggesting that a simple linear model captures a significant, but not complete, portion of the underlying relationship.
- The near-identical performance across regularization strengths suggests the dataset would benefit more from richer features or non-linear models (e.g. polynomial features, tree-based regressors) than from further linear regularization.
- Overall, the experiment demonstrates a complete regression workflow – preprocessing, baseline modeling, regularization, hyperparameter tuning via cross-validation, and rigorous evaluation – and highlights the importance of diagnosing bias vs. variance before deciding on next steps for model improvement.